

Claude Code for System Software & Embedded Engineering - Course Outline

1. Duration

- 16 Hours (2 Days)

2. Objectives

At end of this workshop, participants will be able to :

- Use Claude Code effectively for day-to-day system software engineering work in Linux, C/C++, Python, and shell environments
- Navigate, understand, and safely modify large legacy and open-source codebases
- Select the right Claude model and manage token usage and cost for different classes of engineering tasks
- Build agentic workflows that analyze CI/CD build logs, diagnose failures, and propose or apply fixes
- Apply agentic workflows to identify security vulnerabilities and evaluate patches, with appropriate human checkpoints
- Convert existing engineering knowledge and workflows into reusable Claude Code skills, hooks, and plugins
- Design and orchestrate subagents for automatable, multi-step engineering processes
- Assemble these capabilities into a working proof-of-concept that can be extended after the training

3. Audience

System Software Engineers, Embedded Engineers, and Technical Leads working across new product development, legacy product maintenance, and open-source toolchain management, primarily in Linux, C/C++, Python, and shell-based environments. Participants may also include technically strong Product Managers involved in product maintenance. Prior experience with Claude Code is not required; the program is designed for a mix of first-time and experienced users.

4. Pre-requisite

- Working proficiency in a Linux command-line environment
- Programming or scripting experience in C/C++, Python, or shell scripting
- Working knowledge of Git and CI/CD concepts
- General familiarity with build systems and open-source software components
- No prior experience with Claude Code is required

5. Hardware & Network Requirements

- Desktop/Laptop with minimum 16 GB RAM (Linux native, or Windows/Mac with a Linux environment such as WSL2 or a VM)
- Open Internet connection (minimum 10 Mbps per user)
- Local Admin Access

6. Software Requirements

- Linux environment (native, WSL2, or VM) with a standard C/C++ toolchain (gcc/clang, make, cmake)
- Git Client
- Claude Code CLI (latest version) installed, with a valid Claude subscription (Pro/Max/Team/Enterprise) or API key; each attendee works in their own sandbox repository
- Python 3.10+ and Node.js 18+ (for scripting and MCP server development)
- VS Code IDE or preferred editor (with the Claude Code IDE extension, where available)
- Access to a CI system (e.g., GitHub Actions) for the CI/CD and build-log modules

- Sample open-source codebases will be provided by the trainer for hands-on exercises, since client proprietary code will not be used in the training environment

7. Outline

Day 1

Module-1: Claude Code Fundamentals & Setup

- What Claude Code is and how the agent loop works: built-in tools (Read, Write, Edit, Bash, Grep, Glob), and just enough agentic AI grounding to use it effectively
- Installation, authentication, command-line usage, and IDE integration with VS Code
- Permission modes explained: default (ask before risky actions), Plan Mode (read-only, propose-then-approve), auto-accept edits, and full bypass/auto mode
- Project memory (CLAUDE.md) and slash commands

Hands-on:

- Install and authenticate Claude Code, then clone a small open-source Linux system utility (C/C++ or Python) that will serve as the running example for the two days
- Run the built-in tools against the sample utility and switch between permission modes to observe the approval flow
- Create a starter CLAUDE.md capturing the sample utility's build and coding conventions

Module-2: Claude Code for System Software Development

- Using Plan Mode to turn a requirement into a structured implementation plan for C/C++, Python, or shell code
- Implementing a change with in-line code review, refactors, and diff review
- Hooks for enforcing build and coding conventions at pre/post-tool automation points

Hands-on:

- Plan and implement a small enhancement to the sample utility using Plan Mode
- Review the generated diff and request targeted refactors
- Add a pre-tool hook that enforces one of the utility's coding or build conventions

Module-3: Working with Legacy & Large Codebases

- Strategies for navigating and understanding an unfamiliar or legacy codebase with Claude Code
- Safely modifying code you did not write: dependency awareness, blast-radius analysis, and incremental change
- Automating repetitive engineering tasks across a large codebase

Hands-on:

- Use Claude Code to map the structure and key dependencies of a larger open-source codebase
- Make a small, safe modification to an unfamiliar module and verify it did not break existing behavior
- Automate a repetitive task, such as a consistent code-style or documentation fix, across multiple files

Module-4: AI-Assisted Testing, Debugging & QA

- Test generation and running suites for C/C++, Python, and shell code
- Iterative debugging loops and verification targets
- Automatic checkpoints and restoring state with /rewind

Hands-on:

- Generate and run tests for the sample utility
- Introduce and debug an issue in the same code
- Roll back a change using checkpoints/rewind

Module-5: Model Selection, Token Efficiency & Cost Optimisation

- Context window mechanics, /compact, /clear, /usage, and status line usage tracking
- Choosing a model for the task: fast, lower-cost models for simple, well-scoped changes versus higher-reasoning models for complex analysis or refactors
- Model and effort selection (/model, /effort) and the latency, reasoning, and cost trade-off, illustrated with examples such as a lint fix versus a cross-file root-cause analysis
- Token budgeting for large inputs such as build logs, prompt caching, and delegating verbose work to subagents

Hands-on:

- Run the same task on two different models or effort levels and compare quality, speed, and token cost
- Inspect token usage with /usage and the status line, then compact and clear context to reduce it
- Set a practical rule of thumb for model choice by task type, to reuse in later modules

Day 2

Module-6: CI/CD Build Log Analysis & Automated Troubleshooting

- Connecting Claude Code to CI/CD systems and logs via MCP: architecture, the tools/resources/prompts primitives, and configuring servers via .mcp.json
- Building a custom MCP server that exposes build logs or CI status to Claude Code, tested with the MCP Inspector
- Using Claude Code to analyze a failing build log, identify the root cause, and propose a fix
- Running the triage step non-interactively with headless mode (claude -p) so it can be triggered automatically by the pipeline
- What can safely be automated, such as retriggering a build, versus what should remain a human decision

Hands-on:

- Write and register a custom MCP server exposing the sample utility's CI and build logs
- Test the server standalone with the MCP Inspector, then connect it to Claude Code
- Feed Claude Code a failing build log and have it identify the root cause and propose a fix
- Script the same triage step as a headless invocation, apply the fix, and retrigger the build

Module-7: Agentic Workflows & Sub-Agents

- Defining subagents (.claude/agents) and delegating bounded, high-volume tasks
- Coordinating specialist subagents as an agent team for layered task decomposition
- Chaining an agentic workflow (analyze, decide, act, and trigger a subsequent step), using the CI/CD scenario from Module-6 as the reference pipeline

Hands-on:

- Create a subagent that handles one stage of the build-troubleshooting workflow, such as log triage or fix verification
- Coordinate two subagents so one hands off to the other, and review the isolated output of each

Module-8: Security Vulnerability Analysis & Patch Automation

- Identifying known vulnerabilities in open-source dependencies using common vulnerability databases
- Using Claude Code to analyze a vulnerability, identify an appropriate patch, and evaluate how it can be applied
- Where agentic AI can participate in this workflow, and which steps require human validation and approval

Hands-on:

- Identify a known vulnerability in one of the sample utility's open-source dependencies
- Have Claude Code propose and stage a patch, with an explicit human-approval checkpoint before it is applied
- Apply the patch and verify the fix

Module-9: Skills, Hooks & Plugins for Knowledge Reuse

- Custom skills for reusable domain knowledge: converting an existing engineering process or checklist into a skill
- Hooks for enforcing rules and automating workflow steps at lifecycle events
- Plugins for bundling and distributing skills, subagents, hooks, and MCP servers across a team

Hands-on:

- Turn one of the workflows practiced earlier, such as build-log triage or patch review, into a reusable custom skill
- Add a hook that enforces a rule automatically
- Package the skill, hook, and a subagent into a plugin for reuse across projects

Module-10: Building a Practical Agentic POC

- Assembling MCP access, subagents, skills, and hooks from the two days into one small agentic proof-of-concept
- Setting realistic expectations: a working prototype versus a production-grade solution
- A brief look at the Claude Agent SDK (TypeScript/Python) as the path from a headless prototype to a maintained, unattended agent application
- Practical building blocks participants can adapt to their own internal use cases

Hands-on:

- Combine the CI/CD, security, or codebase-maintenance thread from the two days into a single working POC on the sample utility
- Demo the finished proof-of-concept to the group and capture next steps for extending it internally, including how it could be wrapped with the Agent SDK